



Trabajo Práctico
Análisis de Texto

Fecha de entrega: 09/04/2026

Josue Leonel Gatica Odató 178921

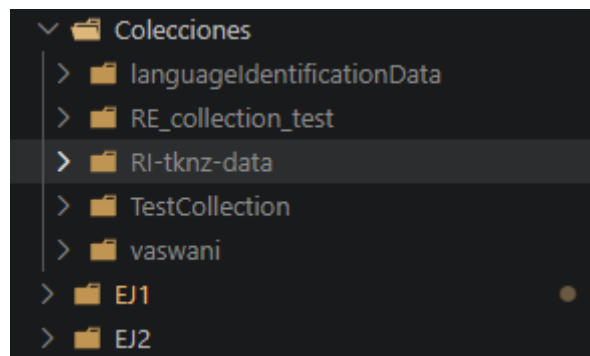
Josuegaticaodato@gmail.com





Aclaración importante sobre colecciones:

Para un correcto manejo de las colecciones en cada ejercicio, los archivos .py tomando las colecciones por defecto en una carpeta llamada “Colecciones”, ubicada a la par con todas las carpetas de los ejercicios.



En ejercicios donde no se indica el pasaje del directorio donde se encuentran los documentos como parámetro (por ejemplo, el EJ1), este toma por defecto la carpeta indicada con el nombre de la colección que se solicitó analizar para ese ejercicio.

Herramienta de IA

El modelo de lenguaje de inteligencia artificial utilizado para el trabajo fue ChatGPT

1) Utilizando la colección para debugging provista por el equipo docente escriba un programa que realice el análisis léxico sobre la misma y extraiga la siguiente información:

- Lista de términos y su DF
- Cantidad de *tokens*
- Cantidad de términos
- Cantidad de documentos procesados

Compare los resultados obtenidos por su programa con los que figuran en el archivo *collection_data.json* que contiene los metadatos de la colección.

Primero, para generar el analizador léxico, recuperamos los tokens que contiene cada texto, pasando toda a minúsculas y eliminando símbolos.

```
1 def normalizar(texto):
2     texto = texto.lower() # Minusculas
3     texto = re.sub(r'^a-záéíóúñ\s', ' ', texto) # Eliminar simbolos
4     tokens = texto.split() # Split por espacio
5     return tokens
```

Luego, realizamos la construcción del índice para cada documento, para obtener la cantidad de apariciones de cada token en ese documento..

```
1 for archivo in archivos:
2     documentos += 1
3
4     # Nombre del archivo a numero
5     doc_id = int(archivo.replace("doc", "").replace(".txt", ""))
6
7     print(os.path.join(ruta, archivo))
8     with open(os.path.join(ruta, archivo), "r", encoding="utf-8") as f:
9         texto = f.read()
10
11     # Obtengo los tokens
12     tokens = normalizar(texto)
13     total_tokens += len(tokens)
14
15     # Construccion del indice
16     for token in tokens:
17         indice[token][doc_id] += 1
18
19     # Agregar palabras al vocabulario (como es conjunto no repite, obteniendo los terminos unicos)
20     vocabulario.update(tokens)
```

Por último, generamos los arreglos correspondientes para docids y frecuencias.

```
1 for termino, docs in indice.items():
2     # Ordenar documentos por docid
3     pares_ordenados = sorted(docs.items())
4
5     docids = [doc for doc, _ in pares_ordenados]
6     freqs = [freq for _, freq in pares_ordenados]
```

Ya con esto, se exporta en formato JSON, siguiente la estructura solicitada.

```
{
  "data": [
    {
      "term": "gato",
      "docid": [...],
      "freq": [...],
      "df": 400
    },
    { ... }
  ],
  "statistics": {
    "N": 10000,
    "num_terms": 20,
    "num_tokens": 793254
  }
}
```

Analizándolo en detalle con el collection_data.json provisto por el equipo docente, solamente se difiere en orden en el que se muestran los términos, ya que no siguen un patrón claro para poder ordenarlo de la siguiente manera y que ambas JSON sean iguales.

Luego, en cuanto a contenido, son exactamente iguales, garantizado la correcta realizando del ejercicio.

Utilización de IA:

Utilizando la herramienta de inteligencia artificial ChatGPT, se realizó lo siguiente:

- Obtención de un lexema para la eliminación de símbolos dentro del texto
- Explicaciones con ejemplos para manejos de índices invertidos y pares ordenados
- Construcción de rutas para lectura correcta de archivos

La validez de la respuesta se verificó en base a pruebas manuales sobre cada fragmento realizado por el chat, validando que los resultados sean los correctos.

Ayudo en el proceso de aprendizaje mediante explicaciones sencillas con ejemplos y pequeños códigos de uso general que serán utilizados en los demás puntos



2) Escriba un programa que realice análisis léxico sobre la colección **RI-tknz-data**. El programa debe recibir como parámetros el directorio donde se encuentran los documentos y un argumento que indica si se deben eliminar las palabras vacías (y en tal caso, el nombre del archivo que las contiene). Defina, además, una longitud mínima y máxima para los términos. Como salida, el programa debe generar:

- Un archivo (`terminos.txt`) con la lista de términos a indexar (ordenado), su frecuencia en la colección y su DF (*Document Frequency*).

Formato de salida: `$<termino> [ESP] <CF> [ESP] <DF>$`.

Ejemplo:

```
casa 238 3
perro 644 6
...
zorro 12 1
```

- Un segundo archivo (`estadisticas.txt`) con los siguientes datos (un ítem por línea y separados por espacio cuando sean más de un valor):
 - Cantidad de documentos procesados.
 - Cantidad de tokens y términos extraídos.
 - Promedio de tokens y términos de los documentos.
 - Largo promedio de un término.
 - Cantidad de *tokens* y términos del documento más corto y del más largo.
 - Cantidad de términos que aparecen sólo 1 vez en la colección.
- Un tercer archivo (`frecuencias.txt`), con:
 - La lista de los 10 términos más frecuentes y su CF (*Collection Frequency*). Un término por línea.
 - La lista de los 10 términos menos frecuentes y su CF. Un término por línea.

Ejecución del programa:



```
python EJ2.py ../Colecciones/RI-tknz-data
```

Construcción del analizador léxico

```
def analizador_lexico(  
    directorio, archivo_stopwords=None, minimo=1, maximo=float("inf")  
):
```

El tokenizer utilizado para realiza lo siguiente:

- Eliminación de acentos
- Eliminar espacios vacíos
- Pasar todos a minúsculas
- Permitir solo términos que estén entre los valores mínimos y máximos

1) Obtención del archivo términos.txt

La salida del archivo términos.txt queda de la siguiente manera (mostrando los primeros 10 términos aproximadamente):

```
de 33057 456  
la 14166 379  
y 10956 366  
en 9908 340  
el 7107 323  
a 5808 353  
que 4659 262  
del 4518 327  
los 3731 312  
las 2995 273  
para 2804 302  
se 2466 266  
universidad 2260 309  
....
```

Al estar ordenado los términos por su frecuencia, podemos identificar la presencia de stopwords o palabras vacías como más frecuentes dentro de todos los documentos. De un total de 498 documentos, el stopwords “de” aparece en 456 de ellos.

2) Obtención del archivo estadísticas.txt

A continuación, se muestra el contenido del archivo estadísticas.txt

```
Cantidad de documentos procesados: 498
Cantidad de tokens extraídos: 367829
Cantidad de terminos extraídos: 30764
Promedio de tokens por documento: 738.6124497991968
Promedio de terminos por documento: 239.60441767068272
Largo promedio de un termino: 8.43388376023924
Cantidad de tokens del documento mas corto: 2
Cantidad de tokens del documento mas largo: 51548
Cantidad de terminos que aparecen solo 1 vez: 15161
```

Lo que puede observar en base a estas estadísticas es lo siguiente:

- La distinción notoria entre termino y token hablada en clase,
- La mitad de los términos extraídos aparecen solo una vez en todos los documentos,
- El largo promedio del token puede variar mucho si se eliminan las stopwords.

3) Obtención del archivo frecuencias.txt

El contenido de frecuencias.txt es el siguiente:

```
Los 10 terminos mas frecuentes y su CF (Collection Frequency):
de 33057
la 14166
y 10956
en 9908
el 7107
a 5808
que 4659
del 4518
los 3731
las 2995

Los 10 terminos menos frecuentes y su CF (Collection Frequency):
tvradio 1
universidadenlace 1
impresoagenda21 1
comunicacioneudem 1
españolversion 1
saspi 1
sistemasgcyt 1
4f3d7b0ddb7c5 1
3725 1
target 1
```

Analizando los 10 primeros términos más frecuentes, puede verse como son todas stopwords que, si se quiere llevar a cabo un correcto análisis, deben ser eliminadas mediante un correcto archivo pasado como parámetro.



Luego, los 10 términos menos frecuentes se muestran como combinaciones de palabras (español y versión, españolversion) o palabras que fueron nombradas solo una vez por el contexto del documento.

3) Escriba un segundo *tokenizer* que implemente los criterios del artículo de Grefenstette y Tapanainen para definir qué es una “palabra” (o término) y cómo tratar números y signos de puntuación. En este caso su *tokenizer* deberá extraer y tratar como un único término:

- Abreviaturas tal cual están escritas (por ejemplo, Dr., Lic., S.A., etc.)
- Direcciones de correo electrónico y URLs.
- Números (por ejemplo, cantidades, teléfonos).
- Nombres propios (por ejemplo, Villa Carlos Paz, Manuel Belgrano, etc.)

Utilice la colección para *debugging* de expresiones regulares provista por el equipo docente para extraer y comparar la salida de su programa con los metadatos de la colección tal como lo realizó en el punto 1.

Por último, extraiga y almacene la misma información que en el punto 2 sobre la colección **RI-tknz-data** utilizando su nuevo *tokenizer*.

En base a los criterios del artículo de Grefenstette y Tapanainen, se construyó el siguiente Tokenizer:

```
token_especificos = [
    ('EMAIL', r"[a-zA-Z0-9][a-zA-Z0-9._%+\-]*@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}"),
    ('URL', r'(?:(https?|ftp?):\/\/[^\s<>"\`']+)',
    ('ABBREV_MULTI', r"(?:[A-Za-záéíóúüñÁÉÍÓÚÜÑ]{1,4}\.){1,}[A-Za-záéíóúüñÁÉÍÓÚÜÑ]{1,4}\.?"", # Abreviaturas S.A. o U.S.A.
    ('ABBREV', r"[A-Za-záéíóúüñÁÉÍÓÚÜÑ]{2,10}\.", # Abreviaturas Dr., Lic.
    ('PHONE', r"[+\\-]?\\d[\\d\\.\\-]*?(?:%|°)?", # Telefonos
    ('NUMBER', r"\\d+(?:[\\.\\d+]*'"),
    ('FECHA', r"\\d{4}[\\-\\/]\\d{1,2}[\\-\\/]\\d{1,2}[\\d{1,2}[\\-\\/]\\d{1,2}[\\-\\/]\\d{4}"",
    ('PROPER_NOUN', r"(?:[A-ZÁÉÍÓÚÜÑ][a-záéíóúüñ+)(?:\\s+(?!Sra\\b|Sr\\b|Dr\\b)[A-ZÁÉÍÓÚÜÑ][a-záéíóúüñ+)+)", # Nombres propios, con varias palabras con mayúscula
    ('WORD', r"[a-záéíóúüñA-ZÁÉÍÓÚÜÑ]{2,}"",
    ('PUNCT', r'[!\\?.,;:]'"),
    ('SIGLA', r"[A-ZÁÉÍÓÚÜÑ]{2,}"")
]

def nuevo_tokenizer(texto):
    tok_regex = '|'.join(f'(?P<{name}>{regex})' for name, regex in token_especificos)

    tokens = []

    for match in re.finditer(tok_regex, texto):
        kind = match.lastgroup
        value = match.group().strip()

        tokens.append(value)

    return tokens
```

El nuevo tokenizer ahora permite emails, abreviaturas, urls, números y nombres propios. Fue necesaria la construcción de una lista de tuplas para cada token específico, vinculándole la expresión regular que más se adecua a ese término.



Se verifico con el collection_data.json que los términos sean tomados de una forma correcta, siguiendo con el análisis sobre la colección RI-tnkz-data

```
python EJ3.py ../Colecciones/RI-tnkz-data
```

```
Cantidad de documentos procesados: 498
Cantidad de tokens extraidos: 367829
Cantidad de terminos extraidos: 30764
Promedio de tokens por documento: 738.6124497991968
Promedio de terminos por documento: 239.60441767068272
Largo promedio de un termino: 8.43388376023924
Cantidad de tokens del documento mas corto: 2
Cantidad de tokens del documento mas largo: 51548
Cantidad de terminos que aparecen solo 1 vez: 15161
```

El resultado para los tres archivos..txt fue el mismo que el ejercicio 1.

Utilización de IA:

Fue necesaria la intervención de la herramienta de IA para poder construir las expresiones regulares correctas basadas en el artículo, con la finalidad de que se realice una correcta tokenizacion sobre los documentos. La comprobación para ello fue el archivo collections_data.json provisto por el equipo docente.



- 4) A partir del programa del ejercicio 1, incluya un proceso de *stemming*. Luego de modificar su programa, corra nuevamente el proceso del ejercicio 2 y analice los cambios en la colección. ¿Qué implica este resultado? Busque ejemplos de pares de términos que tienen la misma raíz pero que el *stemmer* los trató diferente y términos que son diferentes y se los trató igual.

Utilizando la librería de NLTK, utilizamos SnowBallStemmer para hacer el stemming:

```
# Stemming usando SnowballStemmer, libreria de nltk
def stemming(terminos):
    stemmer = SnowballStemmer("spanish")
    return [stemmer.stem(termino) for termino in terminos]
```

Consulta a la IA sobre cómo funciona SnowBallStemmer:

SnowBallStemmer reduce palabras a su raíz aplicando reglas sobre sufijos, específicas para cada idioma. Implementa una evolución del algoritmo de Martin Porter.

Ej. Si la palabra es Nacionalmente

1. Sufijo mente. Como no está en una región valida se borra
Nacionalmente a Nacional
2. Detecta -al, como esta en región valida se deja
Queda finalmente nacional

Este solo soporta el idioma que se le envié como parámetro

Ejecución del programa

```
python EJ4.py ../Colecciones/RI-tnkz-data
```

1) Obtención del archivo términos.txt

```
de 33057 456
la 14166 379
y 10956 366
en 9908 340
el 7107 323
a 5808 353
que 4659 262
del 4523 327
los 3734 312
las 3001 273
par 2842 302
se 2466 266
univers 2436 317
....
```

Puede verse como la palabra “Universidad” fue pasada a “Univers”, aplicando las reglas de Stemming de la librería..

2) Obtención del archivo estadísticas.txt

```
Cantidad de documentos procesados: 498
Cantidad de tokens extraidos: 367829
Cantidad de terminos extraidos: 20931
Promedio de tokens por documento: 738.6124497991968
Promedio de terminos por documento: 239.60441767068272
Largo promedio de un termino: 7.24150781138025
Cantidad de tokens del documento mas corto: 2
Cantidad de tokens del documento mas largo: 51548
Cantidad de terminos que aparecen solo 1 vez: 9869
```

Se puede notar lo siguiente:

- La cantidad de tokens sigue igual, pero la cantidad de términos bajo de 30000 a 2000 aproximadamente, un tercio de los términos fueron pasados a su raíz morfológica.
- Eliminar una tercia parte de los términos hizo que baje el largo promedio de estos, de 8.43 a 7.24 (una letra menos)
- La cantidad de términos que aparecen una vez también bajo de 15000 a 9800.

3) Obtención del archivo frecuencias.txt

```
Los 10 terminos mas frecuentes y su CF (Collection Frequency):  
de 33057  
la 14166  
y 10956  
en 9908  
el 7107  
a 5808  
que 4659  
del 4523  
los 3734  
las 3001  
  
Los 10 terminos menos frecuentes y su CF (Collection Frequency):  
tvradi 1  
universidadenlac 1  
impresoagenda21 1  
comunicacioneudem 1  
españolversion 1  
saspí 1  
sistemasgcyt 1  
4f3d7b0ddb7c5 1  
3725 1  
target 1
```

Los términos más frecuentes no cambiaron, y los términos menos frecuentes fueron afectados por el stemming, aunque las reglas aplicadas no hicieron que se reduzca drásticamente la palabra. Esto último puede deberse a que estamos tratando con palabras combinadas (españolversion por ejemplo, debería de ser tratada por separado, pero aparece junta en uno de los documentos)

Aplicando banco de palabras stopwords:

```
univers 2436 317  
nacional 1586 306  
cienci 1429 191  
estudi 1303 187  
carrer 1298 160  
facult 1267 161  
2012 1248 165  
educ 1200 164  
investig 1158 186  
inform 1114 219
```



```
Cantidad de documentos procesados: 498
Cantidad de tokens extraídos: 367829
Cantidad de terminos extraídos: 20764
Promedio de tokens por documento: 738.6124497991968
Promedio de terminos por documento: 208.1867469879518
Largo promedio de un termino: 7.269408591793488
Cantidad de tokens del documento mas corto: 2
Cantidad de tokens del documento mas largo: 24170
Cantidad de terminos que aparecen solo 1 vez: 9865
```

```
Los 10 terminos mas frecuentes y su CF (Collection Frequency):
univers 2436
nacional 1586
cienci 1429
estudi 1303
carrer 1298
facult 1267
2012 1248
educ 1200
investig 1158
inform 1114

Los 10 terminos menos frecuentes y su CF (Collection Frequency):
tvradi 1
universidadenlac 1
impresoagenda21 1
comunicacioneudem 1
españolversion 1
saspi 1
sistemasgcyt 1
4f3d7b0ddb7c5 1
3725 1
target 1
```

Aplicando un archivo de [stopwords](#) al script, puede verse como el termino más frecuente pasa de 33000 a 2400, además de que el documento más largo tiene 24000 tokens y no 51000.

Ejemplo misma raíz pero el stemmer los trata diferente

Los siguientes 2 términos:

- Rápida
- Rapidez

El stemmer los transformo en:

- Rapid
- Rapidez

Deberían mapearse como Rapid ambos supuestamente.



Ejemplo términos diferentes pero el stemmer los trata igual

Los siguientes 2 términos:

- Universal
- Universidad

Fueron pasados a Univers. Ambos términos no son lo mismo, pero el stemmer lo toma como iguales

Utilización de IA:

- Explicación del funcionamiento SnowBallStemmer, junto con ejemplos
- Ejemplos de pares de términos que podrían estar incluidos dentro de los documentos donde se llevó a cabo el análisis, para poder explicar correctamente los problemas del stemmer



- 5) Sobre la colección Vaswani, ejecute los *stemmers* Porter y Lancaster provistos en el módulo `nltk.stem`. Compare: cantidad de tokens únicos resultantes, resultado 1 a 1 y tiempo de ejecución para toda la colección. ¿Qué conclusiones puede obtener de la ejecución de uno y otro?

Resultados obtenidos:

Corriendo ambos stemmers sobre la colección Vaswani, se obtuvieron los siguientes resultados.

```
Comparacion de Stemmers: Porter vs. Lancaster
-----
Tokens: 479163
Vocabulario: 12189
-----
Cantidad de tokens unicos resultantes:
Porter: 7993
Lancaster:, 6877
-----
Tiempo de ejecucion:
Porter: 8.65088939666748 seg.
Lancaster: 6.020229816436768 seg.
```

Viendo en detalle la cantidad de tokens únicos que generó cada uno, puede verse que Porter generó más de 1000 tokens adicionales que Lancaster, esto que debe que prevalece un enfoque conservador en este stemmer, tratando de no recortar tan bruscamente las palabras y garantizar que las raíces se puedan interpretar mejor. En cambio, Lancaster tiene reglas más agresivas en su algoritmo, reduciendo una gran cantidad de palabras, pero trae consigo el problema de deformar demasiado estas.

En velocidad, es evidente que el enfoque brusco de Lancaster garantiza que el algoritmo realice más rápidamente el stemming, mientras que Porter le toma un tiempo más aplicar sus reglas que favorecen la semántica.



6) Escriba un programa que realice la identificación del lenguaje de un texto a partir de un conjunto de entrenamiento. Pruebe dos métodos sencillos:

- Uno basado en la distribución de la frecuencia de las letras.
- El segundo, basado en calcular la probabilidad de que una letra x preceda a otra letra y (calcule una matriz de probabilidades con todas las combinaciones).

Compare los resultados contra el módulo Python `langdetect` y la solución provista.

Existe 3 archivos:

- `Langdetect-library.py`
Utilizando la librería `langdetect`, realiza la identificación de idiomas
- `EJ6.py`
Utiliza el método de distribución de frecuencias de las letras para identificar idiomas
- `EJ6Matriz.py`
Utiliza el cálculo de la probabilidad de las letras precedentes para identificar idiomas

Ambos métodos se compararon con la solución provista y el resultado en `langdetect`

Método 1 – Distribución de la frecuencia de las letras:

```
Resultados - Distribucion de la frecuencia de las letras
---- COMPARACION CON LA SOLUCION PROVISTA -----
Líneas iguales: 244
Total líneas: 300
Porcentaje: 81.33%
---- COMPARACION CON LANGDETECT -----
Líneas iguales: 243
Total líneas: 300
Porcentaje: 81.00%
```

Método 2 – Probabilidad de letra precedente

```
Resultados - Probabilidad de letras precedentes
---- COMPARACION CON LA SOLUCION PROVISTA -----
Líneas iguales: 282
Total líneas: 300
Porcentaje: 94.00%
---- COMPARACION CON LANGDETECT -----
Líneas iguales: 281
Total líneas: 300
Porcentaje: 93.67%
```



Como puede verse, el segundo método es mejor que el primero para identificar idiomas por 40 líneas más, acercándose el segundo a un porcentaje de acierto del 94%, un numero bastante elevado.

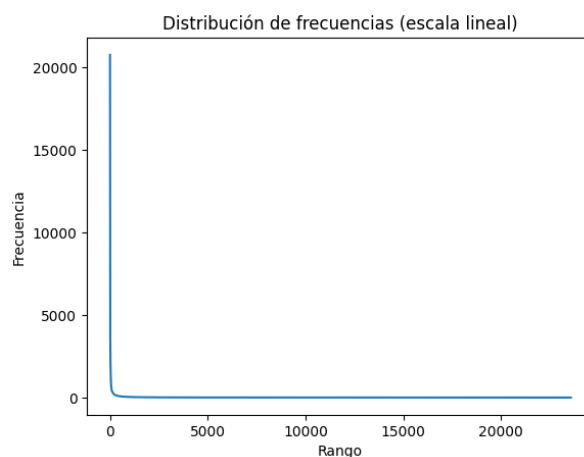
Además, la comparación entre Langdetect y la solución difiere en porcentajes mínimos, donde la librería provista por Python puede equivocarse en 1 línea de las 300 que se probaron.

Utilización de IA:

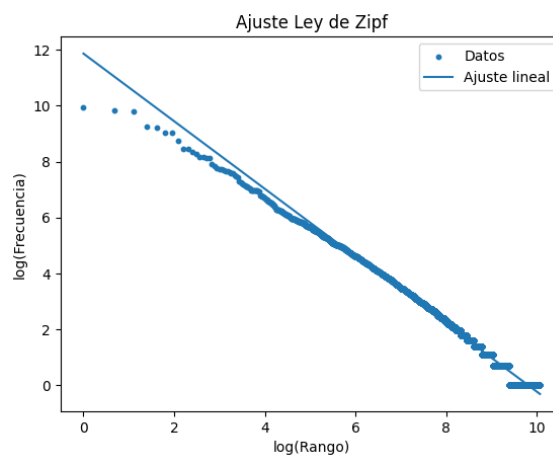
En este caso, la IA sirvió para la construcción de ambos métodos, con una explicación detallada del paso a paso para comprender de la mejor manera ambos algoritmos.

- 7) En este ejercicio se propone verificar la predicción de ley de Zipf. Para ello, descargue desde Project Gutenberg el texto del Quijote de Cervantes y escriba un programa que extraiga los términos y calcule sus frecuencias (el programa debe generar la lista ordenada por frecuencia descendente). Calcule la curva de ajuste utilizando la función *Polyfit* del módulo NumPy. Con los datos crudos y los estimados grafique en la notebook ambas distribuciones (haga 2 gráficos, uno en escala lineal y otro en log-log). ¿Cómo se comporta la predicción? ¿Qué conclusiones puede obtener?

Distribución en escala lineal



Distribución en escala log-log



A través del gráfico en escala log-log, podemos ver como la predicción se ajusta a una línea recta en lo que son el centro de los datos, que coinciden con las palabras que no son ni muy frecuente ni poco comunes.



En este caso, no realizamos la eliminación de stopwords, y eso puede verse en el gráfico de la escala lineal, donde la gran cantidad de frecuencia de palabras se encuentra al principio. Obteniendo los 10 términos más frecuente, tenemos esto:

1. que: 20769
2. de: 18411
3. y: 18272
4. la: 10492
5. a: 9934
6. en: 8285
7. el: 8267
8. no: 6356
9. los: 4769
10. se: 4752

Todas estas son palabras vacías, que sería necesario eliminar para hacer un correcto análisis.

De esta forma, a través del texto del Quijote de Cervantes, obra más destacada de la literatura española, demostramos que la ley de Zypf se cumple en todos los textos literarios.



- 8) Usando los datos del ejercicio anterior y de acuerdo a la ley de Zipf, calcule la cantidad de palabras que debería haber en el 10%, 20% y 30% del vocabulario. Verifique respecto de los valores reales. Utilice esta aproximación para podar el vocabulario en los mismos porcentajes e indique qué porcentaje de la poda coincide con palabras vacías. Extraiga las palabras podadas que no son *stopwords* y verifique si, a su criterio, pueden ser importantes para la recuperación.

Cantidad de palabras por cada porcentaje (%)

Realizamos el cálculo de los tokens que representa el 10%, 20% y 30% del vocabulario, y se obtuvo lo siguiente:

- 10% vocabulario (2357 palabras):
Cobertura: 86.50%
- 20% vocabulario (4715 palabras):
Cobertura: 91.59%
- 30% vocabulario (7072 palabras):
 - Cobertura: 94.09%

Esto nos confirma el comportamiento de la ley de Zipf, donde el 10% del vocabulario representa una gran porción de los tokens, en este caso, el 86% de los mismos. A medida que subimos el porcentaje del vocabulario, la ganancia en la cobertura será menor al igual que la cantidad de palabras.

Poda del vocabulario

Eliminamos las palabras menos frecuentes del vocabulario, teniendo como resultado:

- Poda 90% del vocabulario:
Palabras eliminadas: 21218
- Poda 80% del vocabulario:
Palabras eliminadas: 18860
- Poda 70% del vocabulario:
Palabras eliminadas: 16503

Comparado con las stopwords, que el resultado fue el siguiente:

- Poda 90%:
% stopwords eliminadas: 0.0000
- Poda 80%:
% stopwords eliminadas: 0.0000



- Poda 70%:
% stopwords eliminadas: 0.0000

En todos los casos, la poda de stopwords dio cero. Esto quiere decir que las stopwords se encuentra entre las palabras más frecuentes, haciendo que no sean afectadas por la poda.

Palabras podadas que no son stopwords

Detalle algunas palabras podadas que no son stopwords:

- Conversación
- Santas
- Resurrección
- Embajadas
- Dulces
- Leyéndolos
- Chinche
- Causadora

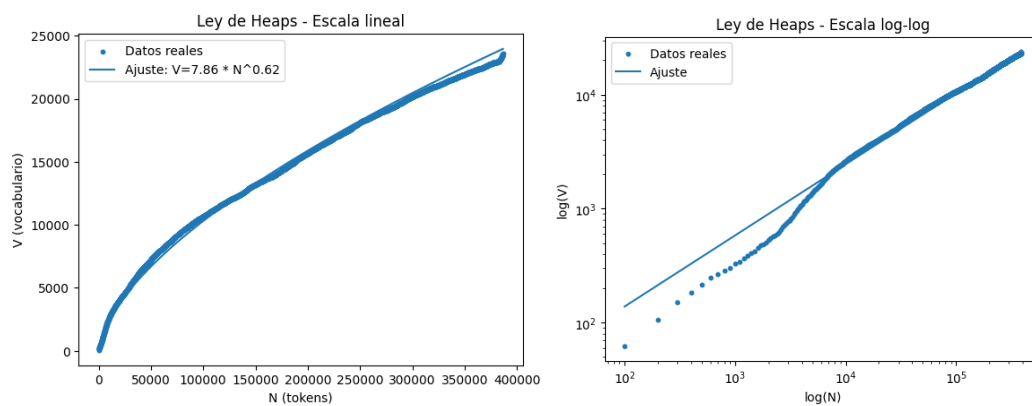
Pueden ser casos que las palabras no sean tan utilizadas por Cervantes en el Quijote, pero esto puede causar un impacto en la recuperación si son términos que solo pertenecen a esta obra literaria, que son relevantes para la búsqueda.

En base a esto último, es necesario analizar el trade-off entre la poda del vocabulario por frecuencia, ya que esta puede mejorar la eficiencia y reducir el ruido, pero afectar la recuperación de la información eliminando términos relevantes en la obra.

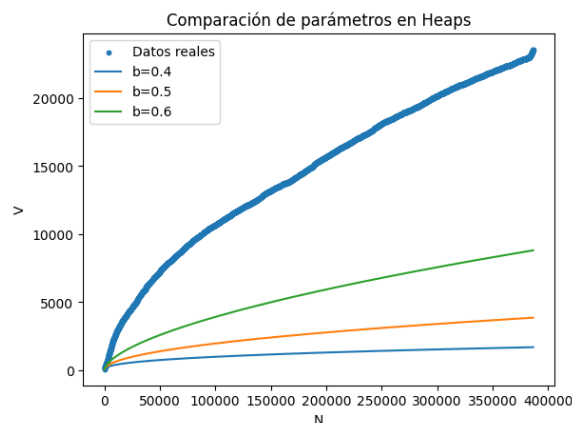
- 9) Codifique un script que reciba como parámetro el nombre de un archivo de texto, tokenize y calcule y escriba a un archivo los pares (#términos totales procesados, #términos únicos). Verifique en qué medida satisface la ley de Heaps. Grafique en la notebook los ajustes variando los parámetros de la expresión. Puede inicialmente probar con los archivos de los puntos anteriores.

Utilizando como archivo de texto nuevamente el Quijote de Cervantes, pero realizo la construcción de los siguientes gráficos a través de los pares terminos totales – terminos unicos

Gráficos lineales y log-log para la ley de Heaps



Como puede verse en la escala lineal, a medida que crece el corpus, se incrementa el número de términos al principio, pero luego se hace más lento. Esto nos indica a la velocidad que crece el vocabulario, donde la aparición de términos nuevos se vuelve logarítmico en cierto punto, hasta llegar a un punto casi asintótico.



En este caso, el valor de B es 0,63 (rango 0.4 a 0.6 valido para verificar la ley de hops) aproximadamente, indicando que el texto es más diverso en cuanto a vocabulario, provocando que la curva no se vuelva asintótica con tanta rapidez. La elección del archivo del texto fue condicionante en este caso, ya que la obra es rica en vocabulario. Comparado con Bs más chicos, donde se vuelve asintótica rápidamente, ya que es más repetitivo.



Bibliografía

- [1] Ricardo Baeza-Yates and Berthier Ribeiro-Neto. Modern Information Retrieval: The Concepts and Technology Behind Search. Addison-Wesley Publishing, USA, 2nd edition, 2008. Cap. 7.
- [2] Gregory Grefenstette and Pasi Tapanainen. What is a word, what is a sentence? problems of tokenization. In Rank Xerox Research Centre, pages 79–87, 1994.
- [3] Le Quan Ha, Darryl Stewart, Philip Hanna, and F Smith. Zipf and type-token rules for the english, spanish, irish and latin languages. Web Journal of Formal, Computational and Cognitive Linguistics, 1 (8):1–12, 1 2006.
- [4] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. Introduction to Information Retrieval. Cambridge University Press, New York, NY, USA, 2008. Cap.6.
- [5] Gabriel Tolosa, Fernando Bordignon. Introducción a la Recuperación de Información. Conceptos, modelos y algoritmos básicos. Laboratorio de Redes de Datos. UNLu, 2005. Cap. 3.
- [6] Daniel Jurafsky & James H. Martin. Speech and Language Processing. January 6, 2026. Cap. 2.